

动态规划 - 分隔等和子集

姓名: 李盛鹏

学号: 2351136

日期: 2025年4月19日

动态规划法

```
class Solution {
public:
    bool canPartition(vector<int>& nums) {
        int sum = std::accumulate(nums.begin(), nums.end(), 0);

        // 如果总和为奇数, 不能分割成两个子集
        if (sum % 2 != 0) {
            return false;
        }

        sum = sum / 2; // 目标子集和

        // 创建 dp 数组, dp[i] 表示是否能找到和为 i 的子集
        vector<bool> dp(sum + 1, false);
        dp[0] = true; // 和为 0 时, 子集为空集

        // 遍历每个数
        for (int num : nums) {
            // 从大到小更新 dp 数组
            for (int i = sum; i >= num; --i) {
                dp[i] = dp[i] || dp[i - num]; // 如果 dp[i - num] 为 true, 就更新 dp[i]
            }
        }

        return dp[sum]; // 如果 dp[sum] 为 true, 说明可以分成两个和相等的子集
    }
};
```

成果

通过 147 / 147 个通过的测试用例

Loving SvansontPq 提交于 2025.04.19 16:13



字节跳动心动计划

面向短期冲刺字节跳动选手, 求职弯道顺利超车

→

🕒 执行用时分布

95 ms | 击败 40.69%

📈 复杂度分析

💾 消耗内存分布

13.28 MB | 击败 81.66%

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

```
// 创建 dp 数组, dp[i] 表示是否能找到和为 i 的子集
vector<bool> dp(sum + 1, false);
dp[0] = true; // 和为 0 时, 子集为空集

// 遍历每个数
for (int num : nums) {
    // 从大到小更新 dp 数组
    for (int i = sum; i >= num; --i) {
        dp[i] = dp[i] || dp[i - num]; // 如果 dp[i - num] 为 true, 就更新 dp[i]
    }
}

return dp[sum]; // 如果 dp[sum] 为 true, 说明可以分成两个和相等的子集
};
```

